

Open in app



Follow

553K Followers



You have **1** free member-only story left this month. [Upgrade for unlimited access.](#)

Serverless BERT with HuggingFace and AWS Lambda

Build a serverless question-answering API with BERT, HuggingFace, the Serverless Framework, and AWS Lambda.



Philipp Schmid Jun 30, 2020 · 7 min read ★



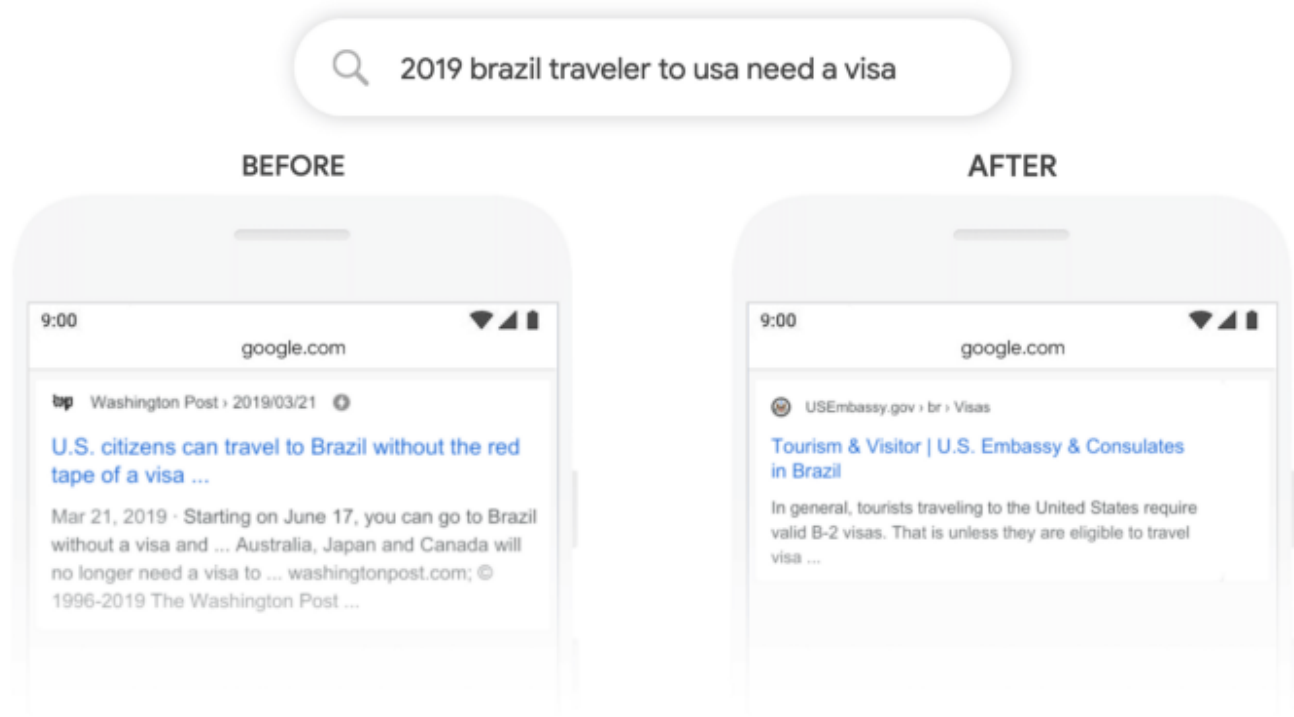
Photo by [Samule Sun](#) on [Unsplash](#)

Originally published at <https://www.philschmid.de> on June 30, 2020.

Introduction

“Serverless” and “BERT” are two topics that strongly influenced the world of computing. Serverless architecture allows us to provide dynamically scale-in and -out the software without managing and provisioning computing power. It allows us, developers, to focus on our applications.

BERT is probably the most known NLP model out there. You can say it changed the way we work with textual data and what we can learn from it. *“BERT will help [Google]. Search [achieve a] better understand[ing] one in 10 searches”*. BERT and its fellow friends RoBERTa, GPT-2, ALBERT, and T5 will drive business and business ideas in the next few years and will change/disrupt business areas like the internet once did.



search language understanding BERT

Imagine the business value you achieve combining these two together. But BERT is not the easiest machine learning model to be deployed in a serverless architecture. BERT is quite big and needs quite some computing power. Most tutorials you find online demonstrate how to deploy BERT in “easy” environments like a VM with 16GB of memory and 4 CPUs.

I will show you how to leverage the benefits of serverless architectures and deploy a BERT Question-Answering API in a serverless environment. We are going to use the Transformers library by HuggingFace, the Serverless Framework, and AWS Lambda.

Transformer Library by Huggingface



[transformers logo by huggingface](#)

The Transformers library provides state-of-the-art machine learning architectures like BERT, GPT-2, RoBERTa, XLM, DistilBert, XLNet, T5 for Natural Language Understanding (NLU) and Natural Language Generation (NLG). It also provides thousands of pre-trained models in 100+ different languages and is deeply interoperability between PyTorch & TensorFlow 2.0. It enables developers to fine-tune machine learning models for different NLP-tasks like text classification, sentiment analysis, question-answering, or text generation.

AWS Lambda

AWS Lambda is a serverless computing service that lets you run code without managing servers. It executes your code only when required and scales automatically, from a few requests per day to thousands per second. You only pay for the compute time you consume — there is no charge when your code is not running.



AWS Lambda Logo

Serverless Framework

The Serverless Framework helps us develop and deploy AWS Lambda functions. It's a CLI that offers structure, automation, and best practices right out of the box. It also allows us to focus on building sophisticated, event-driven, serverless architectures, comprised of functions and events.



Serverless Framework logo

If you aren't familiar or haven't set up the Serverless Framework, take a look at this [quick-start with the Serverless Framework](#).

Tutorial

Before we get started, make sure you have the [Serverless Framework](#) configured and set up. You also need a working Docker environment. A Docker environment is used to build our own python runtime, which we deploy to AWS Lambda. Furthermore, you need access to an AWS Account to create an S3 Bucket and the AWS Lambda function.

In the tutorial, we are going to build a Question-Answering API with a pre-trained `BERT` model. The idea is we send a context (small paragraph) and a question to the lambda function, which will respond with the answer to the question.

As this guide is not about building a model, we will use a pre-built version, that I created using `distilbert`. You can check the colab notebook [here](#).

```
1 context = """We introduce a new language representation model called BERT, which stands
2 Bidirectional Encoder Representations from Transformers. Unlike recent language
3 representation models (Peters et al., 2018a; Radford et al., 2018), BERT is
4 designed to pretrain deep bidirectional representations from unlabeled text by
5 jointly conditioning on both left and right context in all layers. As a result,
6 the pre-trained BERT model can be finetuned with just one additional output
7 layer to create state-of-the-art models for a wide range of tasks, such as
8 question answering and language inference, without substantial task-specific
9 architecture modifications. BERT is conceptually simple and empirically
10 powerful. It obtains new state-of-the-art results on eleven natural language
11 processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute
12 improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1
13 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD
14 v2.0 Test F1 to 83.1 (5.1 point absolute improvement)."""
15
16 question_one = "What is BERT's best score on Squadv2 ?"
17 # 83 . 1
18
19 question_two = "What does the 'B' in BERT stand for?"
20 # 'bidirectional encoder representations from transformers'
```

serverless1.py hosted with ♥ by GitHub

[view raw](#)

Before we start, I want to say that we're not gonna go into detail this time. If you want to understand more about how to use Deep Learning in AWS Lambda I suggest you check out my other articles:

- [Scaling Machine Learning from ZERO to HERO](#)
- [How to Set Up a CI/CD Pipeline for AWS Lambda With GitHub Actions and Serverless](#)

The architecture we are building will look like this.



What are we going to do:

- create a Python Lambda function with the Serverless Framework
- create an S3 Bucket and upload our model
- Configure the `serverless.yaml`, add `transformers` as a dependency and set up an API Gateway for inference
- add the `BERT` model from the [colab notebook](#) to our function
- deploy & test the function

You can find everything we are doing in this [GitHub repository](#) and the [colab notebook](#).

Create a Python Lambda function

First, we create our AWS Lambda function by using the Serverless CLI with the `aws-python3` template.

```
1 serverless create --template aws-python3 --path serverless-bert
```

This CLI command will create a new directory containing a `handler.py`, `.gitignore` and `serverless.yml` file. The `handler.py` contains some basic boilerplate code.

```
1  import json
2
3
4  def hello(event, context):
5      body = {
6          "message": "Go Serverless v1.0! Your function executed successfully!",
7          "input": event
8      }
9      response = {
10         "statusCode": 200,
11         "body": json.dumps(body)
12     }
13     return response
```

serverless3.py hosted with ❤ by GitHub

[view raw](#)

Add `transformers` as a dependency

The Serverless Framework created almost anything we need, except for the `requirements.txt`. We create the `requirements.txt` by hand and add the following dependencies.

```
1  https://download.pytorch.org/whl/cpu/torch-1.5.0%2Bcpu-cp38-cp38-linux_x86_64.whl
2  transformers==2.10
```

serverless4.txt hosted with ❤ by GitHub

[view raw](#)

Create an S3 Bucket and upload the model

AWS S3 and Pytorch provide a unique way of working with machine learning models which are bigger than 250MB. Why 250 MB? The size of the Lambda function is limited to 250MB unzipped.

But S3 allows files to be loaded directly from S3 into memory. In our function, we are going to load our model `squad-distilbert` from S3 into memory and reading it from memory as a buffer in Pytorch.

If you run the [colab notebook](#) it will create a file called `squad-distilbert.tar.gz`, which includes our model.

For creating an S3 Bucket you can either create one using the management console or with this command.

```
1 aws s3api create-bucket --bucket bucket-name --region eu-central-1 --create-bucket-config
```

serverless5.txt hosted with ♥ by GitHub

[view raw](#)

After we created the bucket we can upload our model. You can do it either manually or using the provided python script.

```
1 import boto3
2
3
4 def upload_model(model_path='', s3_bucket='', key_prefix='', aws_profile='default'):
5     s3 = boto3.session.Session(profile_name=aws_profile)
6     client = s3.client('s3')
7     client.upload_file(model_path, s3_bucket, key_prefix)
```

serverless6.py hosted with ♥ by GitHub

[view raw](#)

Configuring the `serverless.yaml`

This time I provided the complete `serverless.yaml` for us. If you want to know what each section is used for, I suggest you check out [Scaling Machine Learning from ZERO to](#)

HERO. In this article, I went through each configuration and explain the usage of them.

```
1  service: serverless-bert
2
3  provider:
4    name: aws
5    runtime: python3.8
6    region: eu-central-1
7    timeout: 60
8    iamRoleStatements:
9      - Effect: "Allow"
10        Action:
11          - s3:getObject
12        Resource: arn:aws:s3:::<your-S3-Bucket>/<key_prefix>/*
13
14  custom:
15    pythonRequirements:
16      dockerizePip: true
17      zip: true
18      slim: true
19      strip: false
20      noDeploy:
21        - docutils
22        - jmespath
23        - pip
24        - python-dateutil
25        - setuptools
26        - six
27        - tensorboard
28      useStaticCache: true
29      useDownloadCache: true
30      cacheLocation: "./cache"
31  package:
32    individually: false
33    exclude:
34      - package.json
35      - package-log.json
36      - node_modules/**
37      - cache/**
38      - test/**
39      - __pycache__/**
40      - .pytest_cache/**
41      - model/pytorch_model.bin
42      - raw/**
```

```

43     - .vscode/**
44     - .ipynb_checkpoints/**
45
46 functions:
47   predict_answer:
48     handler: handler.predict_answer
49     memorySize: 3008
50     timeout: 60
51     events:
52       - http:
53         path: ask
54         method: post
55         cors: true
56
57 plugins:
58   - serverless-python-requirements

```

serverless7.yaml hosted with ❤ by GitHub

[view raw](#)

Add the BERT model from the colab notebook to our function

A typical `transformers` model consists of a `pytorch_model.bin`, `config.json`, `special_tokens_map.json`, `tokenizer_config.json`, and `vocab.txt`. The `pytorch_model.bin` has already been extracted and uploaded to S3.

We are going to add `config.json`, `special_tokens_map.json`, `tokenizer_config.json`, and `vocab.txt` directly into our Lambda function because they are only a few KB in size. Therefore we create a `model` directory in our lambda function.

If this sounds complicated, check out the [GitHub repository](#).

The next step is to create a `model.py` in the `model/` directory that holds our model class `ServerlessModel`.

```

1  from transformers import AutoModelForQuestionAnswering, AutoTokenizer, AutoConfig
2  import torch
3  import boto3
4  import os

```

```

4 import os
5 import tarfile
6 import io
7 import base64
8 import json
9 import re
10
11 s3 = boto3.client('s3')
12
13
14 class ServerlessModel:
15     def __init__(self, model_path=None, s3_bucket=None, file_prefix=None):
16         self.model, self.tokenizer = self.from_pretrained(
17             model_path, s3_bucket, file_prefix)
18
19     def from_pretrained(self, model_path: str, s3_bucket: str, file_prefix: str):
20         model = self.load_model_from_s3(model_path, s3_bucket, file_prefix)
21         tokenizer = self.load_tokenizer(model_path)
22         return model, tokenizer
23
24     def load_model_from_s3(self, model_path: str, s3_bucket: str, file_prefix: str):
25         if model_path and s3_bucket and file_prefix:
26             obj = s3.get_object(Bucket=s3_bucket, Key=file_prefix)
27             bytestream = io.BytesIO(obj['Body'].read())
28             tar = tarfile.open(fileobj=bytestream, mode="r:gz")
29             config = AutoConfig.from_pretrained(f'{model_path}/config.json')
30             for member in tar.getmembers():
31                 if member.name.endswith(".bin"):
32                     f = tar.extractfile(member)
33                     state = torch.load(io.BytesIO(f.read()))
34                     model = AutoModelForQuestionAnswering.from_pretrained(
35                         pretrained_model_name_or_path=None, state_dict=state, config=config)
36             return model
37         else:
38             raise KeyError('No S3 Bucket and Key Prefix provided')
39
40     def load_tokenizer(self, model_path: str):
41         tokenizer = AutoTokenizer.from_pretrained(model_path)
42         return tokenizer
43
44     def encode(self, question, context):
45         encoded = self.tokenizer.encode_plus(question, context)
46         return encoded["input_ids"], encoded["attention_mask"]
47
48     def decode(self, token):

```

```

49         answer_tokens = self.tokenizer.convert_ids_to_tokens(
50             token, skip_special_tokens=True)
51         return self.tokenizer.convert_tokens_to_string(answer_tokens)
52
53     def predict(self, question, context):
54         input_ids, attention_mask = self.encode(question, context)
55         start_scores, end_scores = self.model(torch.tensor(
56             [input_ids]), attention_mask=torch.tensor([attention_mask]))
57         ans_tokens = input_ids[torch.argmax(
58             start_scores): torch.argmax(end_scores)+1]
59         answer = self.decode(ans_tokens)
60         return answer

```

serverless8.py hosted with ♥ by GitHub

[view raw](#)

In the `handler.py` we create an instance of our `ServerlessModel` and can use the `predict` function to get our answer.

```

1  try:
2      import unzip_requirements
3  except ImportError:
4      pass
5  from model.model import ServerlessModel
6  import json
7
8  model = ServerlessModel('./model', <s3_bucket>, <file_prefix>)
9
10
11 def predict_answer(event, context):
12     try:
13         body = json.loads(event['body'])
14         answer = model.predict(body['question'], body['context'])
15
16         return {
17             "statusCode": 200,
18             "headers": {
19                 'Content-Type': 'application/json',
20                 'Access-Control-Allow-Origin': '*',
21                 "Access-Control-Allow-Credentials": True
22             },
23             "body": json.dumps({'answer': answer})
24         }
25

```

```

26     except Exception as e:
27         return {
28             "statusCode": 500,
29             "headers": {
30                 'Content-Type': 'application/json',
31                 'Access-Control-Allow-Origin': '*',
32                 "Access-Control-Allow-Credentials": True
33             },
34             "body": json.dumps({"error": repr(e)})
35         }

```

serverless9.py hosted with ♥ by GitHub

[view raw](#)

Deploy & Test the function

In order to deploy the function you only have to run `serverless deploy`.

After this process is done we should see something like this.

```

Serverless: Stack update finished...
Service Information
service: serverless-qa-bert
stage: dev
region: eu-central-1
stack: serverless-qa-bert-dev
resources: 12
api keys:
None
endpoints:
POST - https://tczd1mvph8.execute-api.eu-central-1.amazonaws.com/dev/ask
functions:
predict_answer: serverless-qa-bert-dev-predict_answer
layers:
None
Serverless: Run the "serverless" command to setup monitoring, troubleshooting and testing.
⚡ Done in 631.81s.

```

Test and Outcome

To test our Lambda function we can use Insomnia, Postman, or any other REST client. Just add a JSON with a `context` and a `question` to the body of your request. Let's try it with our example from the colab notebook.

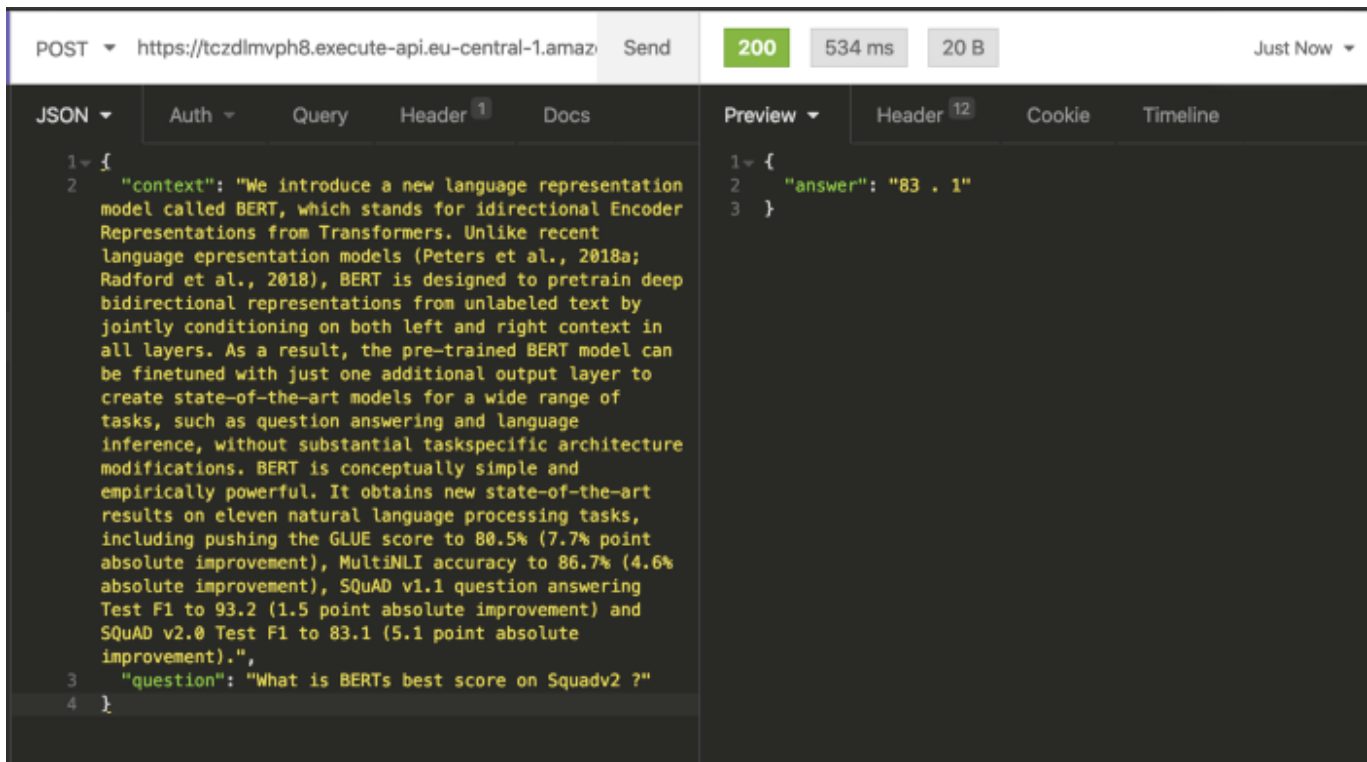
```

1  {
2      "context": "We introduce a new language representation model called BERT, which s
3      "question": "What is BERT?"

```

```
3     "question": "What is BERTs best score on Squadv2 ?"
4 }
```

serverless10.json hosted with ❤ by GitHub [view raw](#)



Our `ServerlessModel` answered our question correctly with `83.1`. Also, you can see the complete request took 319ms with a lambda execution time of around 530ms. To be honest, this is pretty fast.

The best thing is, our BERT model automatically scales up if there are several incoming requests! It scales up to thousands of parallel requests without any worries.

If you rebuild this, you have to be careful that the first request could take a while. First off, the Lambda is unzipping and installing our dependencies and then downloading the model from S3.

Thanks for reading. You can find the [GitHub repository](#) with the complete code [here](#) and the colab notebook [here](#).

Thanks for reading. If you have any questions, feel free to contact me or comment on this article. You can also connect with me on [Twitter](#) or [LinkedIn](#).

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. [Take a look.](#)

Get this newsletter

Emails will be sent to mark@elephantscale.com.

[Not you?](#)

[AWS](#)

[Machine Learning](#)

[Serverless](#)

[Bert](#)

[Deep Learning](#)

[About](#)[Write](#)[Help](#)[Legal](#)

Get the Medium app

